



MÓDULO 2 · EJERCICIO NIVEL INTERMEDIO

Análisis de situaciones · Prompts · Toma de decisiones

Nombre del alumno/a: ANGEL ECHENIQUE

Fecha: 15/03/26

☆☆ Nivel:
INTERMEDIO

INSTRUCCIONES

Nivel intermedio: aplica los principios del módulo a situaciones reales. No hay respuestas de una sola palabra; se espera análisis, justificación y criterio profesional. Tiempo estimado: 50-60 min.



BLOQUE 1

Análisis de principios en situaciones reales

1.1 Diagnóstica: ¿Qué principio falta?

Lee cada situación de trabajo con IA y escribe el principio (o principios) del módulo que el desarrollador está incumpliendo. Justifica tu respuesta en 1-2 frases:

	Situación	Principio(s) incumplido(s)	Justificación breve
1	Marta pide a la IA: «Crea una aplicación de e-commerce completa con pagos, usuarios, carrito, panel admin, emails y API». Recibe 600 líneas de código, no entiende la mitad y no sabe cómo modificarlo.	Itera y refina // Controla el nivel de detalle que queremos	Pedir 600 líneas de golpe es exactamente lo contrario a trabajar por pasos. La IA no puede gestionar bien un proyecto entero en una sola petición.
2	Jaime acepta una función de login generada por la IA y la sube directamente al repositorio porque «compiló sin errores y los tests básicos pasan».	Valida siempre / Mantener el criterio	Que compile y pase tests básicos no significa que sea seguro. Un módulo de login requiere auditoría manual obligatoria.

3	Lucía usa solo GitHub Copilot para absolutamente todo: generar código, buscar documentación, depurar errores y entender conceptos de arquitectura que no conoce.	Yo soy el arquitecto / Aprendo del código generado	La IA no puede tomar decisiones arquitectónicas ni sustituir el conocimiento del programador. Usarla para todo sin criterio genera dependencia y fragilidad.
4	Carlos genera un módulo complejo con IA pero no añade ningún comentario ni documentación sobre qué generó la IA. Tres meses después, nadie del equipo puede mantenerlo.	Documenta decisiones	Sin documentación nadie sabe qué generó la IA ni por qué está estructurado así, lo que hace el código imposible de mantener.
5	Ana recibe una solución de la IA con complejidad $O(n^2)$ para ordenar 100.000 registros. La acepta porque «funciona en los tests con 100 elementos».	Valida siempre / Feedback continuo	Validar solo con datos pequeños es un error crítico. Con 100.000 registros ese algoritmo colapsaría. Hay que analizar la complejidad antes de aceptar.

1.2 El ciclo de feedback

El Principio 9 habla de «feedback continuo». Lee la siguiente conversación con una IA y reescribe cada respuesta del desarrollador aplicando un feedback de calidad profesional:

❓ Un buen feedback es específico, técnico y orienta a la IA hacia lo que necesitas. No basta con decir «no me gusta».

CONTEXTO: Desarrollas una API REST en Node.js para gestionar pedidos de una tienda. Le pides a la IA que genere el controlador de pedidos.



Interacción	Tu feedback mejorado
La IA genera 200 líneas con clases ES6, Mongoose ODM, manejo de errores genérico y sin validaciones de entrada.	Necesito que reescribas el controlador usando funciones puras en lugar de clases ES6, sin Mongoose (usamos PostgreSQL con pg). Añade validación de entrada con Joi para todos los campos del body. El manejo de errores debe ser específico: distingue entre errores de validación (400), no encontrado (404) y error de servidor (500). Máximo 80 líneas por archivo.
La IA refactoriza con Express Router, sigue usando clases, añade try/catch básico pero sigue sin validaciones.	Vas en la dirección correcta con Express Router, pero necesito dos cambios concretos: primero, elimina las clases y usa funciones puras exportadas. Segundo, y es lo más urgente, aún falta la validación de entrada: añade un middleware con Joi que valide que el id del pedido es un UUID válido y que el body contiene los campos obligatorios antes de llegar al controlador.
Nueva versión con funciones puras, añade Joi para validación, pero mezcla lógica de negocio con controlador.	Muy bien con las funciones puras y Joi. Ahora separa la lógica de negocio del controlador: mueve todo lo que no sea recibir la request y devolver la response a un archivo orders.service.js separado. El controlador solo debe llamar al servicio y gestionar la respuesta HTTP. Así seguimos el principio de responsabilidad única

Ahora escribe el feedback mejorado para cada una de las tres interacciones en las celdas de la derecha. Sé específico: indica qué cambiar, por qué y cómo.



BLOQUE 2

Construcción y mejora de prompts profesionales

2.1 Del prompt pobre al prompt profesional

Reescribe cada prompt aplicando el Principio 2 (Especifica el contexto) y la técnica de iteración. Añade: tecnología concreta, restricciones, contexto del proyecto, formato esperado de respuesta.



PROMPT POBRE A

✗ Hazme una función para validar emails.

✓ Tu prompt mejorado:

Estoy desarrollando una API REST en Node.js 20 con Express, creame una función en JavaScript que valide el formato de un email usando regex.

Debe devolver true si es válido y false si no lo es.

Tiene que manejar entradas nulas o undefined sin lanzar errores.

No uses librerías externas.

Añade un comentario explicando el regex utilizado.

PROMPT POBRE B

✗ Necesito autenticación en mi app.

✓ Tu prompt mejorado:

Construyendo una app web con Node.js 20, Express y JWT. Necesito implementar autenticación de usuarios, en este paso solo quiero el middleware de autenticación que verifique el token JWT en el header Authorization.

Debe rechazar con 401 si el token es inválido o ha expirado, y adjuntar el usuario decodificado al objeto req. Usa la librería jsonwebtoken ya instalada en el proyecto.

PROMPT POBRE C

✗ Crea tests para este código.

✓ Tu prompt mejorado: Tengo una función en Node.js que calcula el precio total de un pedido aplicando descuentos y taxes, genera tests unitarios con Jest para esta función.

Cubre al menos estos casos: pedido sin descuento, pedido con descuento del 10%, pedido con tax incluido, array vacío de productos, y valores negativos.

Cada test debe tener un nombre descriptivo en español.



2.2 Diseña la secuencia iterativa

Para construir un sistema de notificaciones por email (registro de usuario, confirmación de pedido, recuperación de contraseña) con Node.js y Nodemailer, diseña una secuencia de 5 prompts iterativos siguiendo el Principio 3:

Paso	Prompt que le enviarías a la IA (sé específico y contextualizado)
1	Estoy en Node.js 20 con Nodemailer ya instalado, creame solo la configuración base del transporter SMTP usando variables de entorno para host, port, user y password. Sin lógica de envío todavía, solo la configuración y exportación del transporter.
2	Usando el transporter anterior, creame una función sendEmail(to, subject, html) genérica que envíe un email y maneje errores de envío. Debe lanzar un error descriptivo si falla y devolver true si tiene éxito. Sin templates todavía.
3	Ahora crea tres funciones específicas que usen sendEmail: sendWelcomeEmail(user), sendOrderConfirmationEmail(user, order) y sendPasswordResetEmail(user, resetToken). Cada una construye su propio HTML básico con los datos recibidos. Sin estilos por ahora.
4	Añade manejo de reintentos a la función sendEmail: si falla, debe intentarlo hasta 3 veces con un delay de 1 segundo entre intentos antes de lanzar el error. Usa async/await sin librerías externas.
5	Genera tests unitarios con Jest para las tres funciones específicas de email. Mockea el transporter de Nodemailer. Cubre el caso de envío exitoso, fallo en el envío y datos de usuario nulos o incompletos.



3.1 Mapa de decisiones: IA vs Manual

Para cada componente del siguiente sistema, decide la estrategia óptima. Justifica tu decisión con al menos una razón técnica:

Sistema: Plataforma de gestión de nóminas para 500 empleados. Stack: TypeScript, NestJS, PostgreSQL, Redis.

	Componente	Estrategia (IA / Manual / Mixta)	Razón técnica principal
1	CRUD de empleados (datos personales, contratos, departamentos)	IA	Es código repetitivo y estructurado, caso ideal para IA. Acelera sin riesgo crítico.
2	Algoritmo de cálculo de retenciones IRPF y seguridad social	MANUAL	La legislación fiscal es precisa y cambia. Un error aquí tiene consecuencias legales y económicas reales para 500 empleados.
3	Boilerplate de endpoints NestJS (DTOs, guards, interceptors)	IA	Los DTOs, guards e interceptors son estructuras repetitivas y bien documentadas. La IA genera código idiomático de NestJS muy bien.
4	Módulo de encriptación y almacenamiento de datos bancarios	MANUAL CON REVISIO EXHAUTIVA	Los datos bancarios están regulados por normativas como PCI-DSS. Un error de seguridad aquí es crítico e irreversible.
5	Generación de informes PDF de nóminas mensuales	MIXTA	La IA genera la estructura y layout del PDF, el programador valida que los datos calculados se representan correctamente.
6	Tests de integración del módulo de cálculo de nóminas	MIXTA	La IA genera la base de los tests, pero el programador debe añadir los casos edge específicos de la lógica fiscal.
7	Migración de esquema de base de datos (datos sensibles reales)	MANUAL	Una migración mal ejecutada sobre datos reales puede ser irreversible. Requiere control total del programador y backup previo.

3.2 Análisis de código generado por IA

La IA ha generado el siguiente endpoint para eliminar un empleado. Identifica todos los problemas que encuentres y propón la solución para cada uno:

```
@Delete('/:id') async
deleteEmployee(
  @Param('id') id: string)
{
  const employee = await this.db.query(
    `DELETE FROM employees WHERE id = ${id}`
  );
  return { deleted: true };
}
```

Nº	Problema detectado	Solución propuesta
1	SQL Injection: el id se inserta directamente en la query sin sanitizar.	Usar consultas parametrizadas: WHERE id = \$1 con el valor como parámetro separado
2	Sin validación del id: no se comprueba si el id es un UUID válido antes de ejecutar.	Añadir un pipe de validación ParseUUIDPipe en el decorador @Param
3	Sin verificación de existencia: si el empleado no existe, devuelve { deleted: true } igualmente.	Comprobar si el registro existe antes de borrar y devolver 404 si no se encuentra
4	Sin manejo de errores: no hay try/catch ni gestión de fallos de base de datos.	Envolver en try/catch y lanzar las excepciones HTTP apropiadas de NestJS.
5	Sin autorización: cualquiera puede llamar a este endpoint y borrar un empleado.	Añadir guard de roles @UseGuards(RolesGuard) para restringir a administradores.



4.1 Dilema profesional

Un cliente te pide entregar un módulo de pagos en 3 días. Con IA puedes generar el 80% del código en pocas horas. Sin IA, necesitarías 2 semanas. El código generado por la IA funciona pero no lo entiendes completamente.

Reflexiona y responde estas tres preguntas:

1. ¿Qué riesgos concretos existen si entregas código que no comprendes completamente?

Si aparece un problema dependerías de la IA para acomodarlos ya que no sabes que hay en el código ni te enteraste de como se llaman las funciones.

2. Aplicando los principios del módulo, ¿qué pasos daría un profesional antes de entregar ese código?

Revisar el código generado por la IA. Que siga la lógica de negocio, muy importante testear todo es probar la app no que lo test pasen eso es un error.
Documentar todo en un archivo aparte donde diga que hace cada bloque de código para futuros bug o aplicación del código.

3. ¿Cómo negociarías los plazos con el cliente manteniendo estándares de calidad? Sé concreto.

Con lo que se ahora mismo, que me de una semana para tenerle el código funcional, testeada y documentada.
Pero le haría pruebas durante una semana mas antes de entregarla, para que luego en producción no tener brechas de seguridad porque eso es crítico.
Luego vendría la entrega del la app y explicación de la misma al cliente para su tranquilidad.